Practical 10

Implementation of Kruskal's algorithm

1. Introduction

Kruskal's Algorithm is a classic greedy algorithm used to find the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph. A Minimum Spanning Tree is a subset of the edges that connects all the vertices together, without any cycles, and with the minimum possible total edge weight.

The algorithm sorts all the edges in increasing order of weight and then adds them one by one to the tree, skipping any edge that would form a cycle. A Disjoint Set Union (Union-Find) data structure is used to check quickly whether two vertices already belong to the same connected component.

2. Aim

To implement Kruskal's Algorithm in C++ using the Union-Find data structure to find the Minimum Spanning Tree of a weighted graph and determine its total weight.

3. Objectives

- To understand the concept of a Minimum Spanning Tree.
- To understand the greedy approach used in Kruskal's Algorithm.
- To understand the Union-Find (Disjoint Set) data structure with path compression.
- To represent a weighted graph as a list of edges.
- To find the edges of the MST and its total weight.
- To analyze the time and space complexity of the algorithm.
- To implement the algorithm using C++.

4. Algorithm

Step 1: Start.

Step 2: Read the number of vertices V, the number of edges E and the list of edges (source, destination, weight).

Step 3: Sort all the edges in increasing order of their weight.

Step 4: Create the parent array and set parent[i] = i for every vertex, so that each vertex is in its own set.

Step 5: Initialize total = 0 and count = 0.

Step 6: Pick the next edge (u, v) from the sorted list.

Step 7: Use find() to get the roots of u and v. If they are different, the edge does not form a cycle, so:

- Print the edge and add it to the MST.
- Merge the two sets using unite(u, v).
- Add the weight of the edge to total and increase count by 1.

Step 8: If the roots are the same, the edge forms a cycle, so skip it.

Step 9: Repeat Steps 6 to 8 until count = V − 1 or all edges are processed.

Step 10: Print the total MST weight.

Step 11: Stop.

5. Pseudocode

```
BEGIN
FUNCTION Find(v)
    IF parent[v] = v THEN RETURN v
    parent[v] ← Find(parent[v])
    RETURN parent[v]
END FUNCTION
FUNCTION Unite(u, v)
    parent[Find(u)] ← Find(v)
END FUNCTION
FUNCTION Kruskal(edges, V, E)
    SORT edges by weight in increasing order
    FOR i ← 0 TO V − 1
        parent[i] ← i
    END FOR
    total ← 0, count ← 0
    FOR i ← 0 TO E − 1 AND count < V − 1
        u ← edges[i].src, v ← edges[i].dest
        IF Find(u) ≠ Find(v) THEN
            PRINT u, v, edges[i].weight
            Unite(u, v)
            total ← total + edges[i].weight
            count ← count + 1
        END IF
    END FOR
    PRINT total
END FUNCTION
END
```

Code:

```
#include <iostream>
#include <algorithm>
using namespace std;

struct Edge {
    int src, dest, weight;
};

bool compare(Edge a, Edge b) {
    return a.weight < b.weight;
}

int parent[10];

int find(int v) {
    if (parent[v] == v)
```

```
        return v;
    return parent[v] = find(parent[v]);
}

void unite(int u, int v) {
    parent[find(u)] = find(v);
}

int main() {
    int V = 4, E = 5;

    Edge edges[] = {
        {0, 1, 10},
        {0, 2, 6},
        {0, 3, 5},
        {1, 3, 15},
        {2, 3, 4}
    };

    sort(edges, edges + E, compare);

    for (int i = 0; i < V; i++)
        parent[i] = i;

    int total = 0;
    cout << "Edges in MST:\n";

    for (int i = 0, count = 0; i < E && count < V - 1; i++) {
        int u = edges[i].src;
        int v = edges[i].dest;

        if (find(u) != find(v)) {
            cout << u << " - " << v
                 << " : " << edges[i].weight << "\n";

            unite(u, v);
            total += edges[i].weight;
            count++;
        }
    }

    cout << "Total MST Weight: " << total;

    return 0;
}
```

Output:

```
Edges in MST:
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10
Total MST Weight: 19
```

The graph has 4 vertices (0 to 3) and 5 edges. After sorting by weight, the edges are 2–3 (4), 0–3 (5), 0–2 (6), 0–1 (10) and 1–3 (15). The algorithm accepts 2–3 and 0–3, rejects 0–2 because it would form a cycle, and then accepts 0–1. The MST has a total weight of 4 + 5 + 10 = 19.

6. Advantages

- Always produces a Minimum Spanning Tree for a connected weighted graph.
- Uses a simple greedy approach that is easy to understand and implement.
- Works very well for sparse graphs.
- Works even if the graph is disconnected (it then gives a minimum spanning forest).
- The Union-Find structure with path compression makes cycle checking very fast.

7. Disadvantages

- Requires sorting all the edges, which takes O(E log E) time.
- Less efficient than Prim's Algorithm for dense graphs.
- Needs the complete list of edges to be stored in memory.
- Works only on undirected, weighted graphs.
- Not suitable for finding shortest paths.

8. Complexity Analysis

Complexity	**Value**
Time Complexity	O(E log E)
Space Complexity	O(V + E)

Where:

- V = number of vertices
- E = number of edges

9. Conclusion

Kruskal's Algorithm was successfully implemented in C++ using the Union-Find data structure. The algorithm sorts the edges by weight and adds each edge to the tree only if it does not form a cycle. It produced the Minimum Spanning Tree with a total weight of 19 for the given graph. Thus, Kruskal's Algorithm provides an efficient and systematic greedy approach for solving the Minimum Spanning Tree problem, especially for sparse graphs.